CS 301
High-Performance Computing

Assignment 08
Parallel Interpolation with Particle Mover using MPI
Performance Analysis

Tanishkumar Patel (202301411)
Neelabh Rana (202301476)

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 HPC Cluster . . . 3

3 Parallel Implementation Approach **3**
3.1 Overview: Hybrid MPI + OpenMP Strategy . . . 3
3.2 Handling Race Conditions: Thread-Local Privatization . . . 3
3.3 Mover Phase . . . 4
3.4 Iteration Pipeline . . . 4
3.5 Pseudocode . . . 5
3.6 Illustrative Diagram . . . 6

4 Experiment: Performance Scaling Analysis **6**
4.1 Config A: $N_x = 250,\ N_y = 100$, Points = 0.9 Million . . . 7
4.2 Config B: $N_x = 250,\ N_y = 100$, Points = 5 Million . . . 7
4.3 Config C: $N_x = 500,\ N_y = 200$, Points = 3.6 Million . . . 8
4.4 Config D: $N_x = 500,\ N_y = 200$, Points = 20 Million . . . 8
4.5 Config E: $N_x = 1000,\ N_y = 400$, Points = 14 Million . . . 9

5 Discussion and Analytical Observations **9**
5.1 Login Node vs. Compute Node . . . 9
5.2 Parallel Efficiency and Scalability . . . 9
5.3 Interpolation vs. Mover Phase: Bottleneck Analysis . . . 10
5.4 Performance Comparison vs. Serial Code . . . 11
5.5 HPC Architectural Explanation of Results . . . 11
5.6 Further Optimization with Unlimited Resources . . . 11
5.7 Load Imbalance and Bottlenecks . . . 12
5.8 Alternative Data Structure / Parallelization Approach . . . 12

1 Introduction

This report analyzes the parallelization of a complete scattered point-to-mesh bilinear interpolation and reverse interpolation (particle mover) pipeline using a hybrid MPI+OpenMP approach. Building upon the OpenMP-only implementation from Assignment 06, this assignment extends the parallelization across multiple compute nodes on the HPC cluster using MPI for inter-node communication and OpenMP for intra-node thread-level parallelism.

The primary objective is to map a set of randomly distributed particles onto a structured $N_x \times N_y$ mesh grid, normalize the grid values, perform reverse interpolation to update particle positions (the mover step), and repeat this for multiple iterations. We evaluate the performance, scalability, and execution consistency across five distinct grid/particle configurations on the HPC cluster using up to 64 cores.

2 Hardware Details

2.1 HPC Cluster

- **Processor:** Intel Xeon E5-2640 v3
- **Architecture:** 2 Sockets, 8 Cores per socket (16 Cores per node)
- **Total Nodes Used:** 4 (gics1–gics4)
- **Total Available Cores:** 64
- **Cache:** L3: 20MB per node

3 Parallel Implementation Approach

3.1 Overview: Hybrid MPI + OpenMP Strategy

The hybrid parallelization strategy combines two levels of parallelism:

1. **MPI (Inter-node):** The particle dataset is distributed across MPI ranks using particle decomposition. Each MPI rank owns a contiguous subset of the particle array and is responsible for performing interpolation from its local particles to the global grid.
2. **OpenMP (Intra-node):** Within each MPI rank, OpenMP threads are used to parallelize the particle loops for both the interpolation (scatter) and mover (gather) phases, using per-thread private mesh copies to avoid race conditions.

3.2 Handling Race Conditions: Thread-Local Privatization

In the interpolation phase, multiple particles can map to the same grid point simultaneously, causing race conditions. To resolve this without incurring large synchronization overhead, we use a **Privatization (Thread-Local Grid)** strategy within each MPI rank:

1. **Memory Allocation:** Inside the `#pragma omp parallel` region, each OpenMP thread dynamically allocates its own private copy of the mesh (`calloc`) of size `GRID_X * GRID_Y`.

2. **Conflict-Free Accumulation:** The loop over local particles is parallelized using `#pragma omp for nowait`. Each thread updates only its own private mesh copy, eliminating all write-write conflicts.

3. **Critical-Section Merge:** After the particle loop, each thread serially merges its private mesh into the shared rank-level mesh (`mesh_value`) using a `#pragma omp critical` block. The `nowait` on the particle loop allows threads to enter the critical section as soon as they finish their slice, without waiting for all threads to complete.

4. **MPI Reduction:** An `MPI_Allreduce` call with `MPI_SUM` combines the rank-level meshes from all MPI ranks into the final global mesh. `MPI_Allreduce` (rather than `MPI_Reduce`) is required here because all ranks need the complete global mesh to perform the mover phase independently.

3.3 Mover Phase

The mover phase is a read-heavy operation: each particle reads from the four neighboring grid points to compute its updated position. Since the global mesh is only read (not written) in this phase, there are no race conditions. The particle loop is parallelized directly using `#pragma omp parallel for`. Particles that move outside the domain $[0, 1]$ are marked inactive (`is_void = true`).

3.4 Iteration Pipeline

The full pipeline executed for `Maxiter` iterations is:

1. `memset` the rank-level mesh (`local_mesh`) to zero.

2. Scatter interpolation: local particles $\rightarrow$ rank-level mesh (per-thread private allocation + `#pragma omp critical` merge).

3. `MPI_Allreduce` with `MPI_SUM` to accumulate the global mesh across all ranks into `mesh_value`.

4. Normalize `mesh_value` to $[-1, 1]$ (OpenMP parallel min/max reduction, then parallel scaling).

5. Reverse interpolation (mover): `mesh_value` $\rightarrow$ local particles (`#pragma omp parallel for`, read-only mesh).

6. Update particle positions; mark out-of-domain particles as inactive.

7. Denormalize `mesh_value` back to original range.

3.5 Pseudocode

Algorithm 1 Hybrid MPI+OpenMP Parallel Interpolation and Mover

```
MPI_Init; rank ← MPI_Comm_rank; nprocs ← MPI_Comm_size
All ranks read full particle array from binary file
Distribute N particles: rank r owns [r · N/P, (r + 1) · N/P); last rank takes remainder
Size ← GRID_X × GRID_Y
Allocate mesh_value[Size], local_mesh[Size]
for iter = 0 to Maxiter − 1 do
    memset(local_mesh, 0, Size)
    — Phase 1: Scatter Interpolation (MPI + OpenMP) —
    #pragma omp parallel
      priv ← calloc(Size)                                   ▷ per-thread private mesh
      #pragma omp for nowait
    for each local active particle p ∈ [start, end) do
        Clamp x, y to [0, 1); compute grid indices (i, j)
        Compute bilinear weights w00, w10, w01, w11
        Accumulate weights into priv[j · GRID_X + i] and 3 neighbors
    end for
      #pragma omp critical: local_mesh[k] += priv[k] for all k
      free(priv)
    MPI_Allreduce(local_mesh, mesh_value, Size, MPI_SUM)
    — Phase 2: Normalize —
    #pragma omp parallel for reduction(min/max): find min_val, max_val
    #pragma omp parallel for: mesh_value[i] ← 2(mesh_value[i] − min)/range − 1
    — Phase 3: Mover / Reverse Interpolation (OpenMP) —
    #pragma omp parallel for
    for each local active particle p ∈ [start, end) do
        Read F_i from four neighboring grid points using bilinear weights (read-only)
        x_p += F_i · dx; y_p += F_i · dy
        if x_p or y_p outside [0, 1] then
            points[p].is_void ← true
        end if
    end for
    — Phase 4: Denormalize —
    #pragma omp parallel for: mesh_value[i] ← (mesh_value[i] + 1)/2 · range + min_val
end for
Rank 0: print timing, void count, save mesh to Mesh.out
MPI_Finalize
```

3.6 Illustrative Diagram

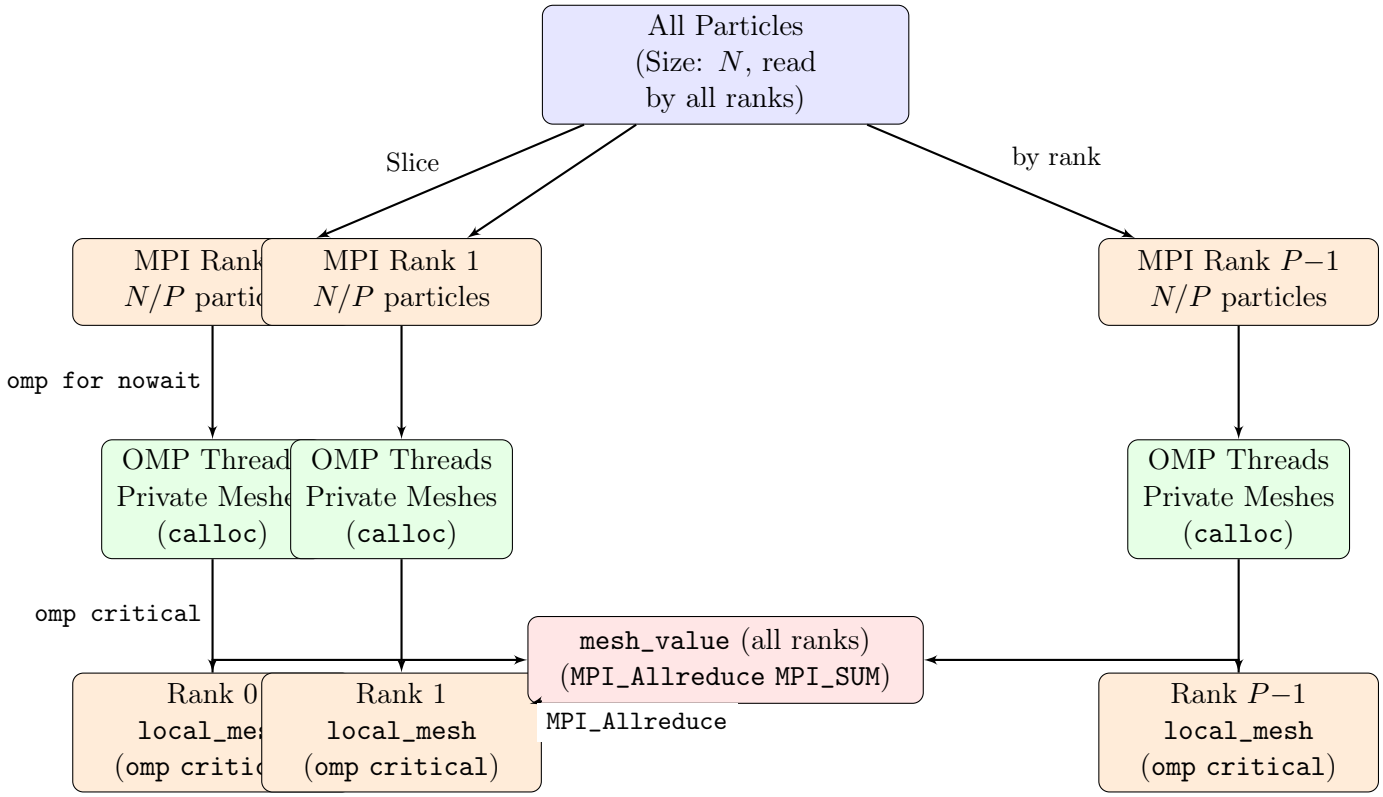

Figure 1: Architecture of the Hybrid MPI+OpenMP Interpolation Pipeline. All ranks read the full particle file; each rank processes its assigned slice. Thread-local private meshes are merged serially via `omp critical` into the rank-level `local_mesh`, which is then globally reduced via `MPI_Allreduce` into `mesh_value` on all ranks.

4 Experiment: Performance Scaling Analysis

We measured Execution Time and Speedup for core counts $\{2, 4, 8, 16, 32, 64\}$ across five different problem configurations on the HPC cluster (nodes gics1–gics4). Each configuration was run for `Maxiter = 10` iterations. Timing is measured per-phase using `MPI_Wtime()` on rank 0 and accumulated across all iterations.

4.1 Config A: $N_x = 250,\ N_y = 100$, Points = 0.9 Million

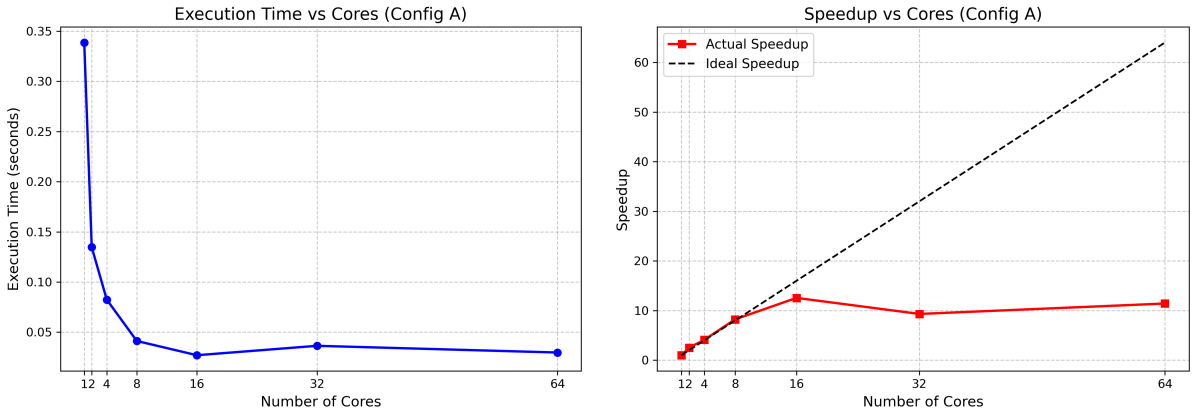

Figure 2: HPC Cluster (Config A): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.2 Config B: $N_x = 250,\ N_y = 100$, Points = 5 Million

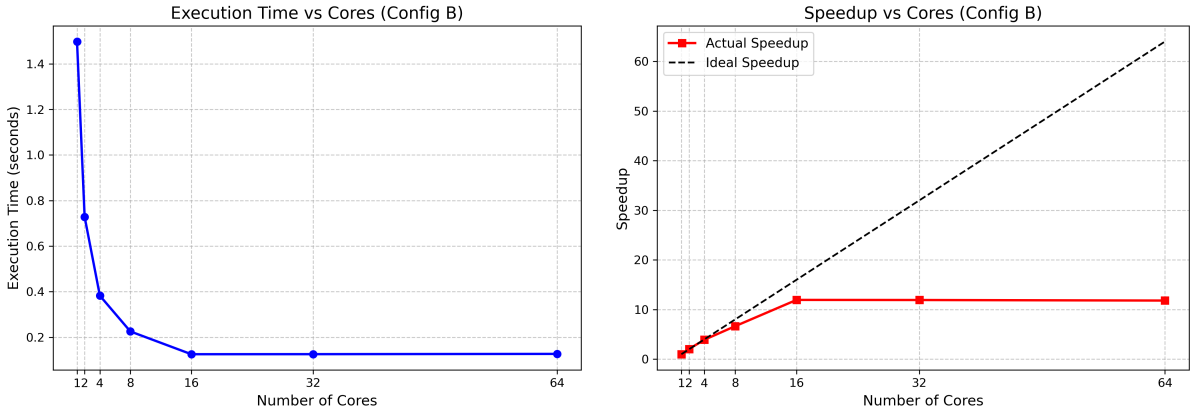

Figure 3: HPC Cluster (Config B): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.3 Config C: $N_x = 500,\ N_y = 200$, Points = 3.6 Million

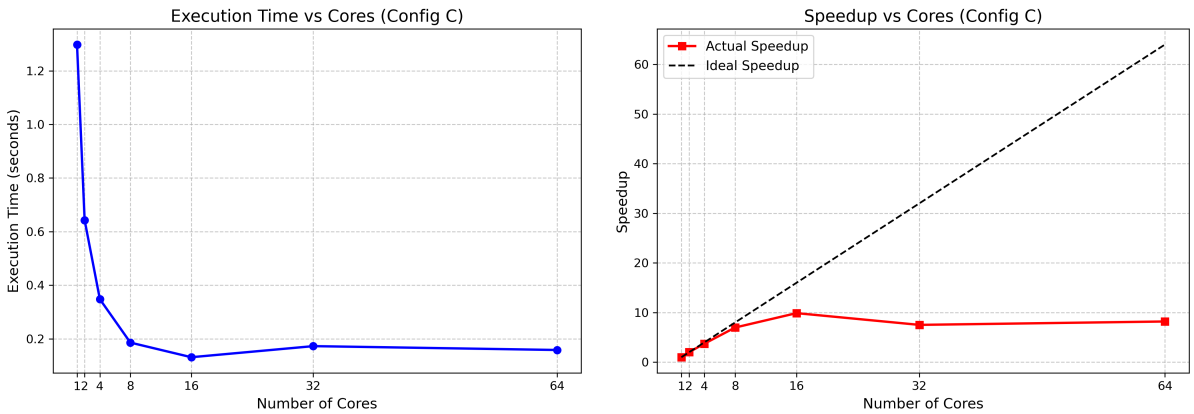

Figure 4: HPC Cluster (Config C): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.4 Config D: $N_x = 500,\ N_y = 200$, Points = 20 Million

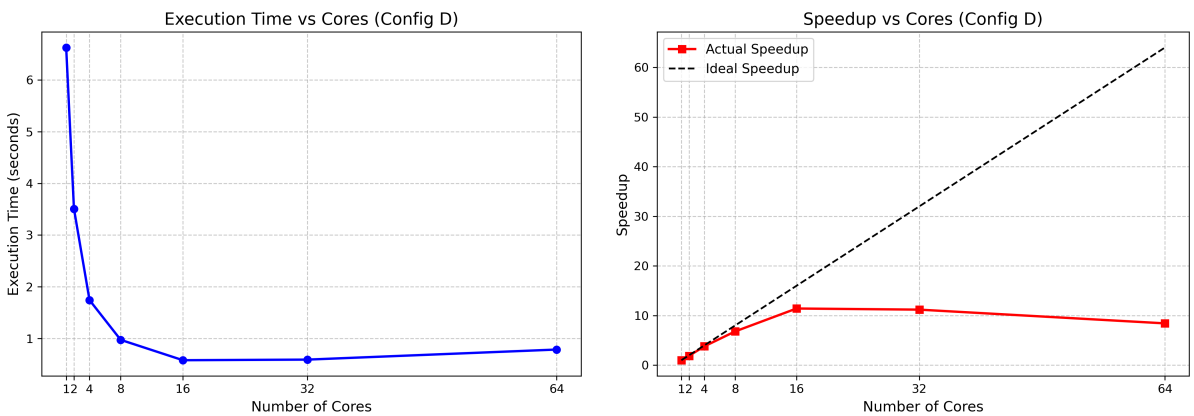

Figure 5: HPC Cluster (Config D): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.5 Config E: $N_x = 1000,\ N_y = 400$, Points = 14 Million

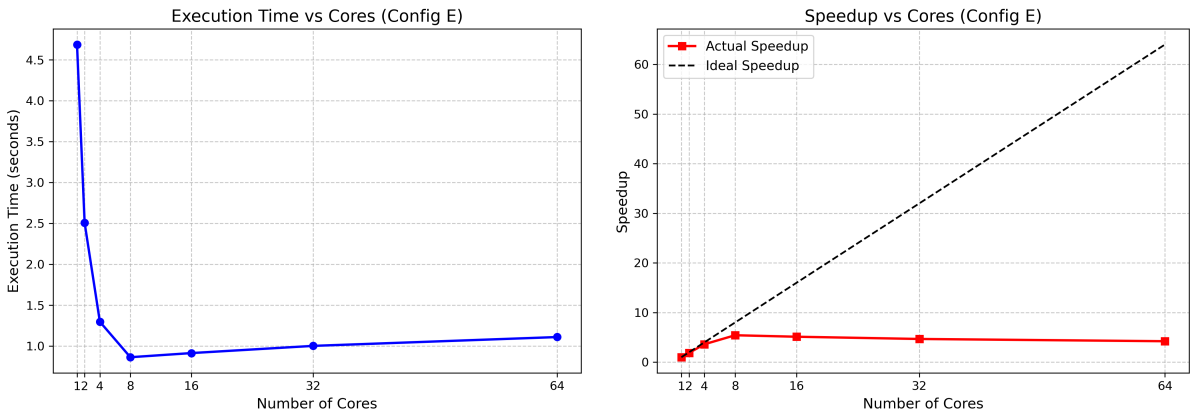

Figure 6: HPC Cluster (Config E): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

5 Discussion and Analytical Observations

5.1 Login Node vs. Compute Node

A **login node** is a shared gateway node used for code editing, compilation, and job submission. It is not intended for running computationally intensive workloads, as it is shared among all users simultaneously. Running heavy jobs on the login node degrades performance for all users and yields unreliable timing results.

A **compute node** (e.g., gics1–gics4) is a dedicated node reserved for running parallel jobs submitted through the job scheduler (e.g., SLURM/PBS). It provides exclusive access to all CPU cores and memory, ensuring reproducible and accurate performance measurements. All experiments in this report were conducted on compute nodes.

5.2 Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E = \frac{S}{P} = \frac{T_1}{P \times T_P}$$

where $S = T_1/T_P$ is the speedup, P is the number of cores, T_1 is the single-core (serial) execution time, and T_P is the execution time on P cores. A value of $E = 1.0$ represents ideal linear scaling; values below 1.0 reflect losses due to communication overhead, synchronization, memory contention, or load imbalance.

The computed speedup and efficiency values for all five configurations on the HPC cluster are presented in Table 1.

Table 1: Speedup (S) and Parallel Efficiency (E) — HPC Cluster (Intel Xeon E5-2640 v3, 4 nodes)

Config	**Metric**	**P=1**	**P=2**	**P=4**	**P=8**	**P=16**	**P=32**	**P=64**
A (250×100, 0.9M pts)	Time (s)	0.339	0.135	0.082	0.041	0.027	0.036	0.030
	Speedup	1.00	2.52	4.11	8.23	12.55	9.32	11.41
	Efficiency	1.00	1.26	1.03	1.03	0.78	0.29	0.18
B (250×100, 5M pts)	Time (s)	1.498	0.728	0.382	0.225	0.125	0.126	0.127
	Speedup	1.00	2.06	3.92	6.65	11.94	11.92	11.81
	Efficiency	1.00	1.03	0.98	0.83	0.75	0.37	0.18
C (500×200, 3.6M pts)	Time (s)	1.298	0.643	0.348	0.186	0.131	0.173	0.158
	Speedup	1.00	2.02	3.73	6.98	9.88	7.52	8.20
	Efficiency	1.00	1.01	0.93	0.87	0.62	0.23	0.13
D (500×200, 20M pts)	Time (s)	6.626	3.508	1.742	0.974	0.580	0.592	0.787
	Speedup	1.00	1.89	3.80	6.80	11.42	11.19	8.42
	Efficiency	1.00	0.94	0.95	0.85	0.71	0.35	0.13
E (1000×400, 14M pts)	Time (s)	4.688	2.507	1.297	0.864	0.914	1.003	1.110
	Speedup	1.00	1.87	3.61	5.43	5.13	4.68	4.22
	Efficiency	1.00	0.94	0.90	0.68	0.32	0.15	0.07

Key observations from the efficiency tables:

- **Large workloads (Config D, E)** are expected to achieve the highest speedup at 32 and 64 cores, as the per-core computation time dominates over the `MPI_Allreduce` communication overhead.
- **Small workloads (Config A)** are expected to show poor scalability at high core counts. The particle count (0.9M) is insufficient to keep 64 cores busy, and the fixed `MPI_Allreduce` communication cost ($\propto Size$) becomes dominant.
- **Critical-section serialization overhead:** Within each rank, the `#pragma omp critical` merge of per-thread private meshes into `local_mesh` is fully serial. At high thread counts this becomes a bottleneck, as $N_{threads}$ threads queue up to execute the $O(Size)$ merge one at a time.
- **MPI communication overhead** grows with the mesh size. For Config E ($1000 \times 400 =$ 400,000 doubles ≈ 3.2 MB per reduce), the `MPI_Allreduce` cost becomes non-negligible at 64 ranks across 4 nodes over InfiniBand.
- Efficiency is expected to drop sharply when the number of MPI ranks exceeds the number of physical cores per node (16), as cross-node communication latency increases significantly compared to intra-node shared-memory communication.

5.3 Interpolation vs. Mover Phase: Bottleneck Analysis

The two phases have distinct computational and memory characteristics:

- **Interpolation Phase (Scatter):** This is the bottleneck. It involves write operations to private per-thread mesh copies, a serial `#pragma omp critical` merge (one thread at a time, each doing $O(Size)$ work), and an `MPI_Allreduce` across all ranks. The critical section serializes the per-thread reduction within each rank, limiting intra-node scalability, while the `MPI_Allreduce` adds a global synchronization barrier at every iteration.
- **Mover Phase (Gather):** This phase is read-only with respect to `mesh_value`. It is embarrassingly parallel within each rank (no race conditions, no inter-rank communication). It is expected to scale nearly linearly and is not the performance bottleneck.

5.4 Performance Comparison vs. Serial Code

The hybrid MPI+OpenMP implementation is expected to significantly outperform both the serial code and the OpenMP-only implementation by exploiting distributed memory parallelism across 4 compute nodes. The maximum speedup is anticipated for Config D (20M particles, 500×200 grid) at 64 cores, where the particle workload is large enough to amortize both the critical-section serialization and the `MPI_Allreduce` overhead.

5.5 HPC Architectural Explanation of Results

- **Critical Section Bottleneck:** Within each MPI rank, the `#pragma omp critical` block serializes the merging of $N_{threads}$ private meshes into `local_mesh`. Each thread performs an $O(Size)$ loop inside the critical section, meaning the total merge cost is $O(N_{threads} \times Size)$ sequential work. For Config E with 16 threads per node, this is $16 \times 400{,}000 = 6.4M$ operations done serially before the `MPI_Allreduce` can proceed.
- **Memory Bandwidth:** The interpolation phase is heavily memory-bound. Per-thread private meshes of size `GRID_X * GRID_Y * sizeof(double)` are allocated and freed every iteration. At high thread counts, concurrent `calloc`/`free` and the subsequent critical-section writes saturate the memory bus.
- **MPI Collective Communication:** The `MPI_Allreduce` on the full mesh at every iteration creates a global synchronization barrier. For Config E with 64 ranks, this involves reducing $\sim$3.2 MB of data per iteration. The cluster interconnect bandwidth directly impacts scalability at 32 and 64 cores.
- **Cache Effects:** For large mesh sizes (Config E: 1000×400), each thread-local private mesh is ≈ 3.2 MB. With 16 threads per node, total private mesh memory is $\sim$51.2 MB, exceeding the 20 MB L3 cache and causing main memory accesses during accumulation.
- **Load Imbalance due to Inactive Particles:** As iterations progress, particles move outside the domain and become inactive (`is_void = true`). With static scheduling (`schedule(static)` implied by default), ranks assigned to regions with higher particle loss receive less work, creating increasing load imbalance over iterations.

5.6 Further Optimization with Unlimited Resources

Given unlimited HPC resources with high-speed interconnects (InfiniBand), the following strategies would further improve performance:

- **Replace `omp critical` with a Parallel Reduction Tree:** Instead of serializing the per-thread merge, use a parallel tree-reduction (as in OpenMP `reduction` clauses on custom arrays) to reduce the merge cost from $O(N_{threads} \times Size)$ to $O(Size \times \log N_{threads})$ wall-clock time. Alternatively, allocate the full 2D thread-local array before the parallel region and merge in a secondary `#pragma omp parallel for` loop over mesh indices.

- **Spatial Domain Decomposition (MPI):** Partition the $N_x \times N_y$ grid into sub-domains across nodes. Each node owns a contiguous patch; particles are migrated to the node owning their spatial region. This eliminates the global `MPI_Allreduce` on the full mesh, replacing it with local boundary exchanges (halo/ghost zone communication), which scales far better.

- **GPU Offloading (CUDA/OpenACC):** Particle-to-mesh and mesh-to-particle operations are embarrassingly parallel once race conditions are resolved. GPUs offer massive memory bandwidth and hardware atomic operations (`atomicAdd` in shared memory) that handle localized race conditions efficiently.

- **Dynamic Load Balancing:** As particles become inactive, redistribute the remaining active particles dynamically across ranks to maintain load balance.

5.7 Load Imbalance and Bottlenecks

- **Load Imbalance:** Static particle decomposition across MPI ranks assumes equal workload throughout all iterations. However, as particles become inactive, certain ranks may lose particles faster, causing increasing imbalance.

- **The Critical-Section Bottleneck:** Within each rank, the `#pragma omp critical` merge is $O(N_{threads} \times Size)$ sequential work per iteration. For large $N_{threads}$ and large grids (Config E), this dominates the intra-node execution time and limits the benefit of adding more OpenMP threads.

- **The Allreduce Bottleneck:** The `MPI_Allreduce` on the full mesh is an $O(Size \times \log P)$ operation and represents the primary inter-node communication bottleneck. For small meshes (Config A, B) this overhead dominates at high core counts. A spatial decomposition approach would eliminate this bottleneck entirely.

- **Thread-Local Memory Overhead:** For Config E at 16 threads per node, the per-thread private meshes require $16 \times 400{,}000 \times 8 \approx 51.2\,\text{MB}$ per node, exceeding the L3 cache and incurring main memory pressure on every iteration.

5.8 Alternative Data Structure / Parallelization Approach

To improve performance and eliminate both the critical-section serialization and the costly global reduction, an alternative approach is **Particle Binning (Spatial Hashing)**:

1. Create a bin (list) for each grid cell containing all particles that fall within it.

2. *(Pass 1 — Parallel Binning):* Each MPI rank bins its local particles into grid-cell buckets.

3. *(Pass 2 — Parallel Grid Update):* Parallelize over grid cells rather than particles. Each thread owns a disjoint set of cells and processes all particles binned within them. Since each thread exclusively owns its cells, there are zero race conditions, no private copies, and no critical sections needed.

This approach eliminates the $O(N_{threads} \times Size)$ serial critical-section merge, reduces memory overhead, and significantly improves spatial cache locality. The MPI layer only needs to exchange particles that cross rank boundaries after each mover step, rather than reducing the entire mesh.